

PAccel: Block-Scaled Low-Bit Weight Containers and the V7 Direct Decode Graph for Dependency-Free On-Device LLM Inference

Cybertwip

PowerEngine Inference Group

Abstract

We present PAccel, a compact container format that stores neural-network weights using block-scaled, symmetric, low-bit (2–8 bit) quantization, together with V7, a direct decode graph that runs a transformer language model from such a container with no graph interpreter and no third-party inference runtime. The format partitions each weight tensor into fixed-size blocks, assigns one half-precision scale per block, and encodes each weight as a mid-rise symmetric code with an implicit zero point, achieving an effective

rate near 4.5 bits per weight at four bits while bounding reconstruction error to half a block step. A name- and shape-aware policy promotes quality-critical tensors to eight bits at negligible byte cost. V7 stores linear and language-model-head weights pre-transposed and reconstructs the compute graph from a small architecture recipe, eliminating the export-and-interpret pipeline and all runtime transposition. We give the exact algebra, prove the round-trip error bound and the packing and tiling bijections, and verify them across independent Go, C++, and Python implementations. We show that PAccel's four-bit block construction is structurally identical to the hardware NVFP4 format, and that on coherent unified-memory accelerators such as NVIDIA's GB10 the V7 layout enables a zero-copy CPU/GPU split that is otherwise unavailable. The reference implementation depends only on the standard library, with no onnxruntime, protobuf, or BLAS.

Keywords: quantization, low-bit inference, model compression, transformer decoding, unified memory, NVFP4.

1 Introduction

Running large language models outside the data center is, before anything else, a memory problem. A model that does not fit in the working memory of a laptop, phone, or desk-side accelerator cannot run regardless of arithmetic throughput, and once it fits, single-stream decoding is bound by how many weight bytes must be read per generated token. Weight quantization addresses both constraints at once, and a precise, portable container for quantized weights is the connective tissue between a downloaded checkpoint and an on-device runtime.

This paper documents PAccel, a weight container and quantization contract, and V7, a direct decode graph that is the optimal route from a PAccel package to a token. Our contributions are: (i) a complete specification of a block-scaled symmetric low-bit format with a half-precision scale table, an AVX-friendly tiled int4 layout, and an automatic mixed-precision policy; (ii) V7, a pre-transposed, recipe-driven decode path that removes the graph interpreter and all runtime transposition; (iii) a proof of the format's numerical invariants verified bit-for-bit across three independent implementations; and (iv) an analysis showing the format aligns structurally with the hardware NVFP4 tensor-core format and with coherent unified-memory accelerators.

2 Background and related work

2.1 Weight quantization

Post-training quantization of LLM weights is well studied. LLM.int8() [3] isolates outlier channels to preserve accuracy at eight bits; GPTQ [4] uses second-order information for accurate low-bit weights; AWQ [5] scales salient channels; and NF4/QLoRA [6] introduce a 4-bit normal-float datatype with block constants. A common thread is *block* or *group* scaling: a single scale shared by a small group of weights bounds the effect of any one outlier. PAccel adopts the simplest effective member of this family — per-block absolute-maximum symmetric quantization — and specifies it exactly so that independent decoders agree to the bit.

2.2 Hardware low-bit formats

Narrow numeric formats have moved into silicon. FP8 [7] is standard on recent accelerators; the Open Compute microscaling formats [9] and NVIDIA's NVFP4 [8] define 4-bit floating types with block-shared scales. NVFP4 in particular groups sixteen 4-bit values under one FP8 scale plus a per-tensor FP32 scale, averaging about 4.5 bits per value. As we show in §7.4, PAccel's four-bit block construction is the same idea expressed in integers.

2.3 Containers and graphs

safetensors [15] is a safe, zero-copy tensor container but stores full-precision weights and no graph. ONNX [16] carries both graph and weights but requires a heavyweight runtime to execute. GGUF with llama.cpp [17] couples a quantized container to a specific engine. PAccel separates concerns: it is a quantized *weight* container with a precise contract, and V7 supplies a minimal, recipe-driven executor for the transformer-decoder family [13, 14], so neither a general graph format nor a general-purpose runtime is needed.

3 The PAccel format

3.1 Block-scaled symmetric quantization

Let a tensor be a flat sequence of weights $w_{0..N-1}$. Quantization uses a bit width $b \in [2, 8]$ and a block size B (default 32). Define $\text{maxCode} = 2^b - 1$ and the center $C = \text{floor}(\text{maxCode}/2)$, the code that represents zero. For each block j the encoder computes the block maximum and a single scale:

```
m_j = max{ |w_i| : i in block j }
s_j = m_j / C (s_j = 1 if m_j = 0)
```

and encodes each weight by rounding to nearest with a centered offset and a clamp:

```
q_i = clamp( round(w_i / s_j) + C, 0, maxCode )
```

Decoding is the exact inverse, $w_i = (q_i - C) \cdot s_j$, requiring only one scale per block and no zero-point table. Rounding is round-half-away-from-zero; this choice is part of the contract because a decoder that rounds differently would produce different codes.

3.2 Packing and the tiled int4 layout

Codes are serialized least-significant-bit first; element i occupies bits $[ib, ib+b)$. The linear payload is $\text{ceil}(Nb/8)$ bytes; the byte-aligned 8-, 4-, and 2-bit cases use fast paths that emit identical bytes. For rank-2 four-bit weights a v6 *output-tiled* layout reorders nibbles into sixteen-lane tiles so a single vector load yields a tile ready for a SIMD product. The reorder is a pure permutation: each nibble appears once and the size is unchanged at $N/2$ bytes.

3.3 Scales and mixed precision

From version 4, block scales are stored as IEEE-754 half precision, halving the scale table; a scale is a single positive magnitude and half precision is ample. A name- and shape-aware policy promotes quality-critical tensors — token embeddings, the LM head, normalization and bias vectors, rotary caches, and any tensor with a dimension above 8192 — to a floor of eight bits, since their quantization error is disproportionately damaging while their share of the byte budget is small.

3.4 Container

A package is the signature PACONNX1, a 28-byte header (version, default width and block size, source size, tensor count), and one self-describing record per tensor carrying its name, source type, encoding (raw, RLE-raw, compressed, or tiled-int4 compressed), shape, per-tensor width and block size, scales, and packed codes. The stream is sequential and skippable, and a header-only read recovers metadata without decoding payloads.

4 The V7 direct decode graph

V7 is a packaging and execution mode rather than a new quantizer. It stores every linear projection and the LM head *pre-transposed* — as [inputs, outputs] rather than the framework [outputs, inputs] — so a forward matmul needs no runtime transpose and no scratch buffer. The compute graph is not serialized; it is reconstructed from a small recipe that records the architecture (model type, layer count, hidden and intermediate widths, head counts, head dimension, vocabulary, RMSNorm epsilon, rotary base, activation, and weight tying).

A compact executor runs the Qwen2/Qwen3, Llama-family decoder directly:

```
h = Embed[token]
for L layers:
  a=RMSNorm(h); q,k,v=a.Wq,a.Wk,a.Wv
  RoPE(q,k); GQA causal attention
  h = h + attn.Wo
  m=RMSNorm(h)
  h = h + (SiLU(m.Wg)*(m.Wu)).Wd
  h=RMSNorm(h); logits = h.Wlm
```

RMSNorm [11], rotary position embedding [10], grouped-query attention, and the SwiGLU feed-forward [12] are the only operations; there is no operator zoo to interpret. Because the recipe plus the direct-layout package fully determine the model, V7 removes the ONNX export, the graph interpreter, and every runtime transpose, giving the smallest resident footprint and the fewest instructions per token the format supports.

4.1 Fused projections

The current layout fuses the projections that share an input. The three attention projections all read the normalized hidden state, so their pre-transposed weights are concatenated along the output axis into one matrix $W_{qkv} = [W_q^T \mid W_k^T \mid W_v^T]$ of shape [hidden, out + 2·kv]; the gate and up projections fuse likewise into [hidden, 2·inter]. A single product then yields all parts, recovered by slicing output columns:

```
x . W_qkv = [ x.Wq^T | x.Wk^T | x.Wv^T ] -> q, k, v
x . W_gate_up = [ x.Wg^T | x.Wu^T ] -> gate, up
```

The fusion is exact — the fused product is bit-identical to the separate products and a column slice recovers each — so it changes performance, not numerics: one matmul replaces three in attention and one replaces two in the MLP, raising arithmetic intensity and leaving a single contiguous tiled-int4 operand per pair. Attention biases, where present, are added to the slices.

5 Error analysis

The encoder is a uniform mid-rise quantizer with step s_j . For any non-clamped weight the reconstruction error is at most half a step, and modelling the rounding residual as uniform on the step interval [1] gives the mean squared error and full-scale

signal-to-quantization-noise ratio:

```
|w_i - w^i_i| <= s_j/2
MSE ~ s_j^2 / 12
SQNR ~ 6.02 b + 1.76 dB
```

Each added bit buys about 6 dB, so the 4-to-8-bit gap is roughly 24 dB — the justification for promoting sensitive tensors in §3.3. Because s_j tracks the local block maximum, a tensor with a few large outliers and many small weights is encoded far more accurately block-by-block than under a single global scale, which is the central reason the format uses per-block rather than per-tensor scaling.

6 Implementation

The reference implementation comprises a Go library for the format and V7 graph, a minimal C++ accelerator carrying the identical quantization algebra plus a portable scalar GEMM, and a command-line frontend that pulls models from Hugging Face, compresses them, and runs an interactive chat. None of the three links onnxruntime, protobuf, Eigen, abseil, or any BLAS; the Go build uses no cgo. The quantizer is deterministic, so identical inputs produce byte-identical packages, which makes cross-language and cross-platform parity testable rather than merely asserted.

7 Evaluation

7.1 Compression

Table 1 gives the effective per-weight rate including the fp16 scale table at the default block size. Mixed precision raises sensitive tensors to eight bits, so a real model lands between its nominal profile and the eight-bit row.

Profile	Codes	Scales	Effective	vs fp32	vs fp16
8-bit	8.0	0.5	8.5 b/w	3.8×	1.9×
4-bit	4.0	0.5	4.5 b/w	7.1×	3.6×
2-bit	2.0	0.5	2.5 b/w	12.8×	6.4×

Table 1. Effective bit rate per weight (b/w), block size 32, fp16 scales.

7.2 Numerical fidelity and parity

We verify four invariants on synthetic and structured inputs across independent Go, C++, and Python implementations of the algebra: the per-element round-trip error stays within half the block scale at 2, 4, and 8 bits; bit packing is a bijection for every width 2–8; stored fp16 scales round-trip within the half-precision step; and the tiled-int4 index map is a permutation that covers every nibble exactly once. All checks pass in every language.

Invariant	Statement	Result
Round-trip	$\text{error} \leq s/2$	pass (2/4/8-bit)
Packing	$\text{unpack} \cdot \text{pack} = \text{id}$	pass (b=2..8)
fp16 scale	within fp16 step	pass
Tile map	nibble permutation	pass

Table 2. Verified invariants (Go, C++, and an independent Python reference).

7.3 A bandwidth-bound throughput model

On a unified-memory accelerator at batch one, decode reads the full weight set per token, so an upper bound on throughput is bandwidth divided by resident weight bytes. Table 3 models a 7B-parameter model on a 273 GB/s pool (the GB10 figure). The numbers are an analytical bound, not a measurement, but they show why the format is the lever: four-bit weights roughly triple to quadruple the bound versus fp16.

Precision	Bytes/param	7B resident	Bound (tok/s)
fp16	2.000	14.0 GB	~19
8-bit PAccel	1.063	7.4 GB	~37
4-bit PAccel	0.563	3.9 GB	~69

Table 3. Analytical batch-1 decode bound at 273 GB/s, including the fp16 scale table. Illustrative upper bound, not measured.

7.4 Hardware alignment: NVFP4 and coherent memory

PAccel's four-bit block-scaled construction is structurally identical to NVFP4 [8]: both group a small run of 4-bit values under a shared higher-precision block scale (NVFP4 uses sixteen values and an FP8 scale; PAccel uses a configurable block with an fp16 scale), and PAccel's v6 tile is sixteen lanes wide, matching the NVFP4 micro-block. A V7 tile therefore

maps almost one-to-one onto an NVFP4 tensor-core operand, requiring at most a scale re-encoding. On the GB10 Grace Blackwell platform [19], whose 20-core Arm CPU and Blackwell GPU share 128 GB of coherent LPDDR5X over NVLink-C2C, V7's single pre-transposed weight image enables a zero-copy split: the eight-bit sensitive tensors run on the CPU while the four-bit blocks run on the NVFP4 tensor cores, both reading the same memory with no transfer. On a discrete GPU behind PCIe such a split is normally a loss; coherent memory makes it free.

8 Beyond text: a shared compute and diffusion core

The format and its dependency-free principle extend to audio, image, and video. A single small **compute engine** supplies the float32 primitives every backbone needs — a biased matrix multiply and fused scaled-dot-product attention — as a cooperative CPU/GPU executor; the standalone build is CPU-only and cgo-free. Above it, a **diffusion core** carries the exact sampler algebra: the scaled-linear and Karras schedules, the Euler v-prediction update used for audio and video, the few-step LCM epsilon update used for image, and classifier-free guidance. With the model output that corresponds to a clean latent, both samplers recover x_0 exactly:

```
Euler: x0 = v(-s/sqrt(s^2+1)) + x/(s^2+1)
LCM: x0 = ( x_t - sqrt(1-a_t) eps ) / sqrt(a_t)
```

A generic loop composes a backbone and a decoder behind interfaces, so the Stable Audio 3 diffusion transformer, the image UNet, and the CogVideoX spatio-temporal transformer — with their VAE decoders, all quantized in the same PAccel format and loaded through the format reader — drop into the same seams. The three engines differ only in pipeline, sampler, and latent geometry, reusing one compute engine and one sampler core.

9 Discussion and limitations

PAccel deliberately fixes a single quantization rule rather than offering a zoo of block types; this is what keeps decoders tiny and bit-exact, but it forgoes data-dependent schemes such as GPTQ-style error compensation, which could be layered on as a pre-pass that only changes the codes, not the container. V7 currently targets the Qwen2/Qwen3, Llama-family decoder; extending the recipe to other decoder variants is additive. The throughput figures here are analytical bounds; an end-to-end measurement on GB10 hardware, and a study of accuracy under the mixed-precision policy across model families, are the natural next steps.

10 Conclusion

We described PAccel, a small block-scaled low-bit weight container with an exactly specified quantization contract, and V7, a pre-transposed, recipe-driven decode graph that removes the graph interpreter and all runtime transposition. The format's invariants are proven and verified across three implementations, its four-bit construction matches the hardware NVFP4 format, and its V7 layout turns the coherent unified memory of modern CPU/GPU accelerators into a zero-copy advantage. The result is a dependency-free path from a downloaded checkpoint to efficient on-device generation.

References

[1] R. M. Gray and D. L. Neuhoff. Quantization. IEEE Trans. Information Theory, 1998.

[2] A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017.

[3] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS, 2022.

[4] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR, 2023.

[5] J. Lin et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys, 2024.

[6] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs (NF4). NeurIPS, 2023.

[7] P. Micikevicius et al. FP8 Formats for Deep Learning. arXiv:2209.05433, 2022.

[8] NVIDIA. Introducing NVFP4 for Efficient and Accurate Low-Precision Inference. NVIDIA Technical Blog, 2025.

[9] Open Compute Project. OCP Microscaling (MX) Formats Specification (MXFP4). 2023.

[10] J. Su et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864, 2021.

[11] B. Zhang and R. Sennrich. Root Mean Square Layer Normalization. NeurIPS, 2019.

[12] N. Shazeer. GLU Variants Improve Transformer. arXiv:2002.05202, 2020.

[13] H. Touvron et al. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971, 2023.

[14] A. Yang et al. Qwen2 Technical Report. arXiv:2407.10671, 2024.

[15] Hugging Face. safetensors: a simple, safe format for storing tensors. 2023.

[16] ONNX Community. Open Neural Network Exchange (ONNX). onnx.ai.

[17] G. Gerganov et al. llama.cpp and the GGUF format. github.com/ggerganov/llama.cpp.

[18] NVIDIA. GB10 Grace Blackwell Superchip and DGX Spark. Product documentation, 2025–2026.